Automatización de pruebas

Todo.ly
Proyectos con token

Caso de prueba TC-API-001 · Sección 2 (30%)
20 de octubre de 2026

Flujo seleccionado

Crear un usuario y, con ese usuario, crear, actualizar y eliminar un proyecto utilizando un token de autenticación. Se añaden consultas para comprobar que los cambios persisten y que el proyecto eliminado no permanece activo.

Repositorio público del código

https://github.com/nohpedro/API-Testing

El repositorio contiene el proyecto Gradle, el escenario Cucumber, las clases Java, el esquema de validación, la evidencia y este documento.

Identificación	Definición
Objetivo	Comprobar el ciclo de vida de un proyecto perteneciente al usuario recién creado, con autenticación por token en todas las operaciones del proyecto.
Tipo y prioridad	Prueba funcional de integración de extremo a extremo. Prioridad alta.
Precondiciones	Todo.ly accesible por HTTPS; JDK compatible; dependencias descargables. No se requiere una cuenta existente.
Datos	Correo único con UUID bajo example.com; contraseña aleatoria; nombre inicial: Proyecto Seccion 2; nombre actualizado: Proyecto Seccion 2 actualizado.
Postcondiciones	Proyecto eliminado y ausente de la lista activa. Cuenta temporal eliminada mediante el hook de limpieza.

Criterio de aprobación

El escenario y sus hooks deben finalizar sin errores. Cada operación debe satisfacer sus aserciones de estado, formato y datos; una respuesta HTTP 200 por sí sola no demuestra que el flujo sea correcto.

Caso de prueba: pasos y resultados

URL base: **https://todo.ly**. Cuerpos JSON y encabezados **Accept** y **Content-Type: application/json**.

N°	Petición / autenticación	Resultado esperado
1	**POST /api/user.json** Sin autenticación. Email, Password y FullName.	HTTP 200; Id positivo; Email y FullName iguales a los datos enviados.
2	**GET /api/authentication/token.json** Basic con el usuario recién creado.	HTTP 200; TokenString presente y no vacío. Guardar el token en memoria.
3	**POST /api/projects.json** Token; Content inicial.	HTTP 200; Id positivo; Content esperado; Deleted = false. Guardar el Id.
4	**GET /api/projects/{id}.json** Token.	HTTP 200; mismo Id; nombre inicial persistido; Deleted = false.
5	**PUT /api/projects/{id}.json** Token; Content actualizado.	HTTP 200; mismo Id; nuevo Content; Deleted = false.
6	**GET /api/projects/{id}.json** Token.	HTTP 200; mismo Id; nombre actualizado persistido; Deleted = false.
7	**DELETE /api/projects/{id}.json** Token.	HTTP 200; mismo Id; nombre actualizado; Deleted = true.
8	**GET /api/projects.json** Token.	HTTP 200; lista JSON que no contiene el Id del proyecto eliminado.
9	**DELETE /api/user/{userId}.json** Hook @After; Basic de la cuenta temporal.	HTTP 200; Id igual al usuario creado. La limpieza también se comprueba.

Validaciones transversales

Las respuestas de objetos deben ser JSON y no contener ErrorCode ni ErrorMessage con valor. Las respuestas de proyecto cumplen un esquema mínimo: Id entero positivo, Content de texto no vacío y Deleted booleano. Los campos adicionales son válidos.

El resultado esperado de eliminación es HTTP 200 con un objeto marcado como eliminado. La implementación valida ese contrato concreto de Todo.ly.

Escenario ejecutable y diseño clean

```
@Seccion2 @Regression @Token
Feature: Gestion de proyectos con token en Todo.ly
  Como usuario nuevo de Todo.ly
  quiero gestionar un proyecto con mi token
  para verificar su creacion, actualizacion y eliminacion.

  Scenario: TC-API-001 Crear usuario y gestionar un proyecto con token
    Given creo un usuario nuevo de prueba
    And obtengo su token de autenticacion
    When creo el proyecto "Proyecto Seccion 2" usando el token
    Then el proyecto creado se llama "Proyecto Seccion 2"
    And al consultar el proyecto su nombre es "Proyecto Seccion 2"
    When actualizo el nombre a "Proyecto Seccion 2 actualizado" usando el token
    Then el proyecto conserva su identificador y se llama "Proyecto Seccion 2 actualizado"
    And al consultar el proyecto su nombre es "Proyecto Seccion 2 actualizado"
    When elimino el proyecto usando el token
    Then el proyecto eliminado se llama "Proyecto Seccion 2 actualizado" y esta marcado como eliminado
    And el proyecto ya no aparece en la lista de proyectos activos
```

Todo.ly recibe el encabezado **Token: <TokenString>**. La autenticación Basic se utiliza para obtener el token y para eliminar la cuenta temporal; las peticiones de proyecto llevan exclusivamente el token.

Separación de responsabilidades

Capa	Responsabilidad
Feature / runner	Gherkin expresa el comportamiento. RunCucumberTest descubre el escenario con JUnit Platform.
Steps / contexto	ProjectSteps coordina acciones y hooks. ScenarioContext conserva los identificadores y el token sólo durante el escenario.
Cliente HTTP	ApiClient centraliza transporte, autenticación y timeout. TodoApi define rutas y cuerpos.
Validación	ResponseChecks concentra aserciones comunes; schemas/project.json verifica estructura y tipos.

Adaptación del repositorio de clase

Se conserva la separación entre escenario, pasos y cliente del ejemplo crudProjectClean. Se reemplazan las credenciales fijas y la autenticación básica del CRUD por un usuario temporal y su token. Las peticiones se crean de nuevo en cada llamada; no hay autenticación global ni estado estático mutable.

Evidencia de ejecución real

APROBADO · 1 escenario · 11 pasos · 2 hooks sin fallos

Ejecución: 20/10/2026 09:45:24 (America/La_Paz).
Entorno: Windows, OpenJDK 27 y Gradle Wrapper 9.8.0.
Comando: **.\gradlew.bat clean test --no-daemon**.

Operación registrada	Autenticación	HTTP	Tiempo
POST /api/user.json	sin autenticacion	200	1217 ms
GET /api/authentication/token.json	Basic	200	646 ms
POST /api/projects.json	Token	200	628 ms
GET /api/projects/4558948.json	Token	200	629 ms
PUT /api/projects/4558948.json	Token	200	626 ms
GET /api/projects/4558948.json	Token	200	730 ms
DELETE /api/projects/4558948.json	Token	200	622 ms
GET /api/projects.json	Token	200	632 ms
DELETE /api/user/834064.json	Basic	200	822 ms

Qué demuestra esta ejecución

Los pasos y las validaciones se ejecutaron contra el servicio real. Las llamadas del proyecto registraron autenticación Token; las verificaciones de contenido, identidad, esquema, persistencia y ausencia de la lista activa pasaron. El hook completó la eliminación de la cuenta temporal.

Evidencia reproducible

Los resultados estructurados están en docs/evidence/: cucumber.json, cucumber.xml, cucumber.html y execution.json. El JSON conserva el estado de cada paso y hook, además de los registros HTTP mostrados en esta página.

Los tiempos son mediciones de esta ejecución y no un umbral de rendimiento. Esta evidencia cubre el flujo positivo seleccionado; no demuestra cobertura de casos negativos, carga o seguridad integral.

Incidencia durante la preparación: una ejecución previa dejó una cuenta temporal porque la ruta de limpieza respondió 404. Se corrigió para incluir el Id. La cuenta anterior no pudo recuperarse porque sus credenciales no se guardaron; la ejecución final aquí documentada sí eliminó su cuenta.

Reproducción y referencias

Preparar y ejecutar

Instalar un JDK compatible con Gradle 9.8.0 y configurar JAVA_HOME. Se validó con OpenJDK 27; Java 8 no sirve para estas dependencias. La primera ejecución necesita Internet para descargar Gradle y las bibliotecas.

```
git clone https://github.com/nohpedro/API-Testing.git
cd API-Testing
.\gradlew.bat clean test

# Alternativa por etiqueta, como en el ejemplo de clase:
.\gradlew.bat clean cucumberCli -Psuite="@Seccion2"

# Linux / macOS:
./gradlew clean test
```

Abrir build/reports/cucumber/cucumber.html para consultar los pasos y build/reports/tests/test/index.html para el resultado de Gradle. La tarea clean borra los resultados generados de la ejecución anterior.

Versiones fijadas

Gradle 9.8.0; Cucumber 8.0.4; Rest Assured y JSON Schema Validator 6.0.1; JUnit Platform 6.0.0; Jackson Databind y módulo JDK8 2.22.0. El código se compila con nivel de lenguaje Java 17.

Limpieza y protección de datos

La contraseña y el token se generan u obtienen en ejecución y permanecen en memoria. Los reportes incluyen método, ruta, modalidad de autenticación, estado y tiempo; no imprimen credenciales ni cuerpos sensibles. Si falla el flujo, @After intenta eliminar el proyecto pendiente y después la cuenta. Una caída del servicio durante la limpieza puede impedir la eliminación remota.

Relación con los materiales proporcionados

api-aut.pdf, páginas 5-11: se aplican los métodos HTTP, encabezados y cuerpos JSON; se verifican código, contenido, esquema y efectos persistentes.
cucum.pdf, páginas 3-10: se aplican BDD, Gherkin, definiciones de pasos, etiquetas y hooks.
api_v6-main.zip: se toma como referencia build.gradle, el paquete client, MyStepdefsClean y crudProjectClean.feature.

Referencias consultadas

Todo.ly API Wiki - recursos User, Projects y Authentication.
Gradle 9.8.0: matriz de compatibilidad de Java.
Materiales de clase facilitados para esta actividad. Consulta: 20/10/2026.